Students Marks Analyzer

```python
class StudentMarksAnalyzer:
    def __init__(self, marks_dict):
        self.marks_dict = marks_dict

    def average_marks(self):
        averages = {}
        for student, marks in self.marks_dict.items():
            averages[student] = sum(marks) // len(marks)
        return averages

    def find_topper(self):
        averages = self.average_marks()
        topper = max(averages, key = averages.get)
        return topper, averages[topper]

    def below_thresold(self, thresold = 40):
        averages = self.average_marks()
        return [student for student, avg in averages.items() if avg < thresold]

students = {
    "Shreeram": [39, 40, 35],
    "Krishna": [81, 82, 95],
    "Suraj": [82, 83, 96],
    "Chinmay": [83, 84, 97],
    "Ganesh": [85, 86, 98]
}

analyzer = StudentMarksAnalyzer(students)
print("Average Marks:", analyzer.average_marks())
print("Topper:", analyzer.find_topper())
print("Students below 40:", analyzer.below_thresold(40))
```

Output:

Average Marks: {'Shreeram': 38, 'Krishna': 86, 'Suraj': 87, 'Chinmay': 88, 'Ganesh': 89}

Topper: ('Ganesh', 89)

Students below 40: ['Shreeram']

- **Data Structures**
  - **Dictionary (**marks_dict**):** Maps student names (keys) to a list of marks (values).
  - **List (**marks**):** Stores individual marks for each student.
  - **Derived dictionary (**averages**):** Maps student names to their average marks.
- **Algorithms**

1. **Average Calculation**: Iterates through each student's marks, computes sum, divides by count.
2. **Topper Identification**: Uses max() with dictionary values to find the highest average.
3. **Below Threshold**: Filters students whose average is less than the threshold.

◆ **Complexity**

- **Average Marks**:
  - Time: $O(n \cdot m)$, where n= number of students, m= average number of marks per student.
  - Space: $O(n)$for storing averages.
- **Find Topper**:
  - Time: $O(n)$(scans averages once).
  - Space: $O(n)$(reuses averages).
- **Below Threshold**:
  - Time: $O(n)$.
  - Space: $O(k)$, where k= number of students below threshold.

Overall, the analyzer is efficient: **linear time complexity** relative to the number of students and marks, with **minimal extra space**.